

Auditr Code and Process Explanation

Detailed architecture, runtime flow, model pipeline, and file-by-file explanation based on the current cleaned frontend/backend structure.

1. Project purpose

Auditr is a Streamlit-based audit intelligence workspace for accounting ledgers. It accepts CSV files inside named audit projects, validates and cleans them, engineers fraud-related features, scores suspicious transactions using XGBoost, adds anomaly-assisted prioritization, and then shows the results through an auditor-focused workflow.

2. Clean architecture

- `app.py` is only a thin root launcher.
- `frontend/app_shell.py` controls the Streamlit shell and page routing.
- `frontend/pages` contains the UI pages that the auditor sees.
- `backend/utls.py` contains the real application engine.
- `backend/models/xgb_model.py` contains the readable Python definition of the deployed XGBoost model configuration.
- `backend/models/artifacts` contains the saved trained artifact and supporting metadata files.
- `backend/training` contains the scripts used to generate data and train/evaluate the model.

3. End-to-end runtime process

- 1 The user launches the app through `Launch Auditr.bat` or `streamlit run app.py`.
- 2 `app.py` calls `frontend.app_shell.run_app()`.
- 3 `frontend/app_shell.py` sets the Streamlit config, restores state, injects styling, and calls `render_auth_gate()`.
- 4 After login, the user navigates mainly between Home, Projects, Dashboard, Transactions, and Explainability.
- 5 Projects is the core workflow page where the auditor creates a project and uploads a ledger CSV.
- 6 `backend/utls.py` reads the raw CSV, tries multiple encodings and separators, normalizes column names, and builds a schema profile.
- 7 `prepare_ledger_for_analysis()` validates the incoming file and quarantines bad rows such as invalid dates, invalid amounts, or missing vendors.
- 8 `run_audit_analysis()` preprocesses the valid rows, builds the model feature matrix, loads the trained model artifact, computes fraud probabilities, adds anomaly scores, computes a blended priority score, and generates explanations.
- 9 The resulting analysis bundle is stored in session state and also cached per project to speed up refresh and reopen actions.
- 10 Dashboard, Transactions, and Explainability all consume the same active-project analysis object so the entire workspace follows the selected engagement.

4. Frontend files explained

app.py

This file is intentionally very small. Its only role is to call the real frontend shell. This keeps the root entrypoint clean.

```
from frontend.app_shell import run_app

run_app()
```

frontend/app_shell.py

This is the UI shell. It sets page configuration, initializes state, restores browser persistence, runs the authentication gate, builds the menu popover, and routes between the main page render functions.

frontend/pages/Home.py

Home is the landing page. If no project is active, it shows a clean empty state and directs the auditor to Projects. If a project is active, it becomes an engagement cockpit showing project summary, key metrics, top cases, and vendor watchlist.

frontend/pages/Projects.py

Projects is the most important workflow page. It lets the user create a project, upload a ledger, preview column mapping, see the data-quality and quarantine summary, open an existing project, delete projects, or load the full demo pack.

frontend/pages/Dashboard.py

Dashboard converts the analysis into an auditor-first overview. It shows review counts, why-flagged patterns, department concentration, vendor exposure, control signals, the manual review queue, and extra risk-intelligence visuals such as the heatmap and relationship network.

frontend/pages/Transactions.py

Transactions is the full ledger explorer. It exposes filters, search, risk thresholding, reviewer feedback updates, and report downloads.

frontend/pages/Explainability.py

Explainability focuses on one flagged case at a time. It shows the risk score, main reason, case facts, signal table, vendor evidence, reviewer decision, and a hidden advanced contribution chart.

5. Backend files explained

backend/utils.py

This is the central engine of Auditr. It contains authentication logic, theme helpers, session-state helpers, SQLite project persistence, CSV parsing, column alias mapping, quarantine handling, feature engineering, anomaly scoring, XGBoost inference, explanation generation, chart creation, export helpers, and audit memo generation.

- `read_ledger_csv()` tries multiple encodings and delimiters to support messy real-world CSV inputs.
- `prepare_ledger_for_analysis()` validates the upload before scoring and produces a schema profile plus quarantine table.
- `preprocess_ledger()` creates historical and control-based features from the raw transactions.

- `compute_anomaly_scores()` uses Isolation Forest when possible and falls back to normalized z-score logic otherwise.
- `run_audit_analysis()` performs inference, contribution extraction, blended scoring, and reviewer-facing summary generation.
- `explain_transaction()` builds the detailed single-case explanation used by the Explainability page.

backend/models/xgb_model.py

This file exists so you can show readable model code in viva. It defines the deployed XGBoost parameter set, exposes all candidate benchmark models, and provides helper functions to load and save the trained artifact bundle.

The important point is that the `.pkl` file is only the trained saved artifact. The real code definition of the deployed XGBoost configuration is now visible in `xgb_model.py`.

backend/training/train_auditr_model.py

This script prepares the labeled training data, builds the feature matrix, benchmarks multiple model families, applies a chronological holdout split, chooses the threshold, and exports the trained production model bundle plus metadata.

backend/training/generate_auditr_training_data.py

This script generates the synthetic labeled training dataset used for experimentation and demonstration. It creates normal patterns first and then injects fraud patterns such as duplicate invoices, high-amount anomalies, split payments, department shifts, and payment-method switches.

backend/demo/build_graduation_demo_pack.py

This script builds the smaller curated demo CSV pack used for viva and poster demonstrations, including high-risk focus files, messy CSV files, alias-header files, and locale-format files.

6. Detailed CSV intake process

CSV robustness is one of the strongest parts of the project. The system does not assume perfect input. It first attempts to read the file using multiple encodings such as UTF-8, CP1252, and Latin-1, and multiple separators such as comma, semicolon, tab, and pipe.

After reading, it deduplicates repeated headers, normalizes column names, checks which required fields are available, applies alias mapping, and fills safe placeholders for optional missing fields.

Then it attempts to parse dates and amounts. Rows with invalid dates, invalid amounts, or missing vendors are not mixed into the model input. Instead, they are placed into a quarantine table that can be downloaded and corrected separately. Only the clean rows continue into the scoring pipeline.

7. Feature engineering process

The project relies on structured tabular features built from transaction history rather than deep learning embeddings. This is more appropriate for accounting ledgers and easier to explain.

- Amount behavior features: vendor average amount, amount deviation, signed deviation, vendor amount z-score, and amount-to-vendor-average ratio.
- Invoice behavior features: invoice count, invoice gap days, duplicate invoice flag, near-duplicate similarity, and fuzzy invoice match count.
- Burst and control features: transactions per day per vendor, vendor daily running totals, just-below-approval indicator, and split-payment threshold crossing.

- Behavioral novelty features: new vendor-department pair, new vendor payment method, and new vendor account type.
- Timing features: posting hour, posting weekday, weekend flag, days since last vendor payment, and days since last employee payment.

8. Model and anomaly process

The main classifier is XGBoost. It predicts `fraud_probability` for each transaction. The app then compares that probability with a stored decision threshold from the model metadata to determine `fraud_prediction`.

Anomaly scoring is computed separately. The function `compute_anomaly_scores()` first tries Isolation Forest if the dependency is available and enough rows exist. If that is not possible, the code falls back to z-score normalization across the numeric feature set.

The final queue ordering is not based only on the fraud probability. The code also computes `blended_risk_score` using `fraud_probability`, `anomaly_score`, and a small control-intensity component. This becomes `blended_priority_score`, which is then used to sort the manual review queue.

```
blended_risk_score = 0.66 * fraud_probability + 0.26 * anomaly_score + 0.08 *
control_intensity
```

9. Explainability process

The app uses the XGBoost booster to compute per-feature contribution values. These contributions are stored in a contribution frame. For each transaction, the code selects the primary driver and converts it into a plain-English explanation, summary title, and suggested next step.

This means the auditor does not only see a risk score. The auditor also sees what pattern influenced the score and what evidence should be checked next.

10. Training and evaluation process

The training pipeline uses a chronological split rather than a random split. That is important because in finance data the ordering of transactions matters, and random splits can leak future patterns into past training.

- 70% training data
- 15% validation data
- 15% test data

According to the saved metadata, the current XGBoost model achieved a test ROC-AUC of 0.989 and a test F1 score of 0.937 under chronological holdout evaluation.

After evaluation, the final deployment model is retrained on all available rows and saved as a bundle for the live app. This is why the app uses a `.pkl` artifact at runtime, but the readable configuration and training logic remain visible in Python files.

11. Storage and persistence

Auditr stores project metadata and case feedback in SQLite under `auditr_workspace/auditr.db`. Uploaded ledgers, data-quality reports, quarantined rows, and per-project analysis caches remain stored as local project files inside `auditr_workspace/projects`.

12. What to show in viva

- Show backend/models/xgb_model.py if they ask where the XGBoost model is defined.
- Show backend/training/train_auditr_model.py if they ask how the model was trained and evaluated.
- Show backend/utils.py if they ask how CSV validation, feature engineering, scoring, anomaly detection, and explanation work.
- Show frontend/pages/Projects.py and frontend/pages/Explainability.py if they ask how the user workflow is implemented.